

```
    if(remainder != 0):  
❶         return(gcd(smaller, remainder))
```

[Listing 2-1:](#) *Implementing Euclid's algorithm using recursion*

The first thing to notice is that we don't need any of the $q_1, q_2, q_3 \dots$ quotients. We need only the remainders, the successive letters of the alphabet. Remainders are easy to get in Python: we can use the `%` operator from the previous section. We can write a function that takes the remainder after division for any two numbers. If the remainder is zero, then the greatest common divisor is the smaller of the two inputs. If the remainder is not zero, we use the smaller of the two inputs and the remainder as inputs into the same function.

Notice that this function calls itself if the remainder is nonzero ❶. The act of a function calling itself is known as *recursion*. Recursion can seem intimidating or confusing at first; a function that calls itself may seem paradoxical, like a snake that can eat itself or a person trying to fly by pulling on their own bootstraps. But don't be scared. If you're unfamiliar with recursion, one of the best things to do is start with a concrete example, like finding the greatest common divisor of 105 and 33, and follow each step of the code as if you are the computer. You will see that in this example, recursion is just a concise way to express the steps we listed in “Doing Euclid's Algorithm by Hand” on page 20. There is always a danger with recursion that you create an infinite recursion—that a function calls itself, and while calling itself, calls itself again, and nothing ever causes the function to end, so it attempts to call itself endlessly, which is a problem because we need the program to terminate in order to get the final answer. In this case, we can feel safe because at each step we are getting smaller and smaller remainders that will eventually go down to zero and enable us to exit the function.

Euclid's algorithm is short and sweet and useful. I encourage you to create an even more concise implementation of it in Python.